

BRAIN

Metodología de Gestión del Conocimiento

Asistida por Inteligencia Artificial

Convierte grandes corpus de documentación técnica, normativa o estratégica en grafos de conocimiento atómico consultables por IA — con trazabilidad completa, cumplimiento verificable y generación de código conforme.

ELABORADO POR

Sebastian Dominguez

VERSIÓN

1.0 · Junio 2026

ESTADO

Publicado

Índice

- 01 Qué es BRAIN**
Definición, el problema que resuelve
- 02 Para quién aplica**
Dominios: regulatorio, técnico, estratégico, legal, operativo
- 03 Los 4 pilares**
Atomizar · Conectar · Curar · Capa IA
- 04 Estructura del vault**
Carpetas, frontmatter, wikilinks, MOCs
- 05 Lo que produce**
Respuestas, cumplimiento, código conforme, documentos PDF
- 06 Pipeline de generación de documentos**
matplotlib + Typst + atribución
- 07 Principios de diseño**
Fidelidad, atomicidad, citación, enlace liberal
- 08 Caso de aplicación**
Ejemplo ilustrativo hipotético
- 09 Autoría e IP**
© 2026 Sebastian Dominguez

01 — Qué es BRAIN

BRAIN es una metodología para convertir grandes corpus de documentación — técnica, normativa, estratégica, legal u operativa — en **grafos de conocimiento atómico consultables por IA**, capaces de generar respuestas precisas, verificar cumplimiento y producir código o documentos estructurados.

El problema

Los documentos monolíticos no escalan para trabajo asistido por IA.
Un corpus de 50,000+ líneas distribuido en decenas de archivos presenta tres problemas fundamentales:

Problema	Consecuencia
Contexto excede ventana de IA	El sistema no puede leer todo el corpus a la vez; genera respuestas sin contexto completo
Sin estructura semántica	La IA no puede razonar sobre relaciones entre reglas, flujos y errores
Sin trazabilidad	No hay forma de verificar de dónde viene cada afirmación o decisión

La solución

BRAIN convierte esa masa documental en una **red de nodos atómicos**: una nota por concepto, una nota por regla, una nota por flujo. Todos interconectados con enlaces semánticos y etiquetados con frontmatter estructurado.

- El resultado es un “segundo cerebro” que:
- Responde preguntas complejas **citando la fuente exacta**
 - Verifica si una decisión **cumple las reglas del dominio**
 - Sirve como contexto preciso para que una IA **genere código conforme**
 - Produce **documentos estructurados** consolidando el conocimiento acumulado

02 — Para quién aplica

BRAIN es aplicable a cualquier dominio con **alta densidad documental y requisitos de consistencia**:

Dominio	Tipo de corpus	Lo que produce
Regulatorio / fintech	Reglamentos, circulares, especificaciones técnicas	Cumplimiento verificable, código conforme a norma
Legal / compliance	Contratos, políticas, marcos regulatorios	Consultas rápidas, identificación de obligaciones
Estratégico / producto	Research, análisis de mercado, roadmaps	Decisiones informadas, documentos de producto
Técnico / ingeniería	APIs, specs, arquitecturas, runbooks	Código generado con contexto correcto
Operativo	Procesos, procedimientos, manuales	Consulta rápida, automatización de flujos

Cuándo es más valioso

La metodología es especialmente valiosa cuando se cumplen uno o más de estos criterios:

- El corpus supera las **10,000 líneas** y no cabe en un solo contexto de IA
- Las reglas y los datos **evolucionan con frecuencia** (versiones, actualizaciones normativas)
- Se necesita **trazabilidad**: toda respuesta debe citar su fuente
- **Múltiples personas o sistemas** consultan el mismo conocimiento
- Se requiere generar **código o documentos** que reflejen las reglas del dominio

03 — Los 4 pilares

01 Atomizar

Partir cada documento fuente en **unidades mínimas de conocimiento**. Una nota atómica contiene exactamente una idea: un concepto, una regla, un flujo, un mensaje, un código de error.

Criterio de atomicidad: si la nota trata dos cosas que podrían cambiar de forma independiente, debe partirse en dos.

Cada nota lleva: frontmatter YAML con tipo, ID, etiquetas y estado · cuerpo estructurado en lenguaje natural · sección de implicaciones para implementación (en flujos y procesos) · cita de fuente al pie con archivo y página de origen.

02 Conectar

Enlazar cada nota con todas las notas relacionadas usando **wikilinks** (`[[nombre-nota]]`). Los enlaces son el tejido conectivo del grafo:

- Una regla enlaza a los flujos que la aplican
- Un flujo enlaza a los mensajes que usa, las reglas que cumple y los errores posibles
- Un concepto enlaza a todos los contextos donde aparece

Un enlace a una nota que aún no existe es **válido** — marca trabajo futuro sin romper el grafo. Esto permite construir el sistema incrementalmente.

03 Curar

Mantener la calidad del grafo a lo largo del tiempo:

- **MOC (Maps of Content):** índices temáticos que agrupan notas por dominio
- **Glosario:** una nota por término del dominio, con definición canónica y alias para términos equivalentes
- **Fusión de duplicados:** cuando dos notas representan el mismo concepto, se fusionan con alias
- **Ciclo de retroalimentación:** todo aprendizaje nuevo vuelve al cerebro como nota nueva o actualización

04 Capa IA

El grafo curado se convierte en el **contexto preciso** para sistemas de IA:

- **Consultas en lenguaje natural:** la IA navega los enlaces y responde citando las notas
- **Verificación de cumplimiento:** la IA sigue el árbol de enlaces desde un flujo hasta sus reglas
- **Generación de código:** la sección “Implicaciones para implementación” de cada nota es el checklist de requisitos
- **Generación de documentos:** las notas estratégicas se consolidan en PDFs estructurados

04 — Estructura del vault

Un vault BRAIN tiene la siguiente estructura genérica, adaptable a cualquier dominio:

```
BRAIN/  
├─ README.md      ← plan maestro y estado  
├─ CLAUDE.md      ← convenciones para IA  
├─ HOME.md        ← dashboard / punto de entrada  
├─  
├─ 00-MOC/        ← índices temáticos  
├─ 01-Conceptos/  ← glosario atómico  
├─ 02-Flujos/     ← procesos y flujos  
├─ 03-Reglamentos/ ← reglas y obligaciones  
├─ 04-Tecnico/    ← APIs, mensajes, specs  
├─ 05-Errores/    ← catálogo de errores  
├─ 06-Seguridad/  ← lineamientos de seguridad  
├─ 07-UX/         ← experiencia de usuario  
├─ 08-MVPs/       ← especificaciones de código  
├─ 09-Estrategia/ ← análisis y oportunidades  
├─  
├─ _templates/    ← plantillas de nota  
└─ Markdown/      ← fuentes originales (NO MODIFICAR)
```

Frontmatter estándar

Cada nota lleva frontmatter YAML mínimo:

```
—  
tipo: flujo      # concepto | flujo | regla | tecnico | estrategia  
id: FLUJO-EJEMPLO-01  
tags: flujo, proceso, ejemplo  
fuente: "[[nombre-documento-fuente]]"  
estado: documentado # borrador | documentado | verificado  
—
```

05 — Lo que produce

Un vault BRAIN bien construido produce cuatro tipos de salidas:

01 — Respuestas citadas

Cualquier pregunta sobre el dominio se responde con citas a las notas fuente. La IA entra por el MOC del tema, navega los enlaces, responde con el dato exacto y cita la nota como fuente.

02 — Verificación de cumplimiento

Para cualquier flujo o proceso, la IA navega desde la nota hasta sus reglas enlazadas y produce una lista de verificación: cumple / no cumple / pendiente.

03 — Código conforme

La sección “Implicaciones para implementación” en cada nota de flujo lista los requisitos técnicos. La IA usa esa sección + los enlaces a mensajes, reglas y errores para generar código que ya cumple las especificaciones del dominio.

04 — Documentos estructurados

Las notas estratégicas y de análisis se consolidan en documentos PDF mediante el pipeline de generación descrito en la Sección 06 — con diseño profesional, gráficos y atribución completa.

06 — Pipeline de generación de documentos

BRAIN incluye un pipeline para convertir el conocimiento acumulado en documentos PDF entregables — con identidad visual consistente, atribución de autoría y trazabilidad a las notas fuente. Es un método reproducible, no un diseño ad-hoc por documento.

Principio rector: el markdown es la fuente de verdad; el PDF es una salida derivada. El PDF nunca se escribe a mano: se compila desde un archivo `.typ` que refleja el markdown. Si el contenido cambia, cambia el markdown primero y luego se regenera.

Stack

Herramienta	Para qué	Cuándo
Typst (0.14+)	Motor de tipografía → PDF. 10× más rápido que LaTeX, sintaxis tipo CSS	Por defecto
Python 3 + matplotlib	Gráficos PNG para incrustar (≥180 DPI, paleta de marca)	Si hay gráficos
HTML + Chrome headless	Alternativa para diseños muy visuales (--print-to-pdf)	Opcional

Flujo de generación

- 1 Consolidar** — Las notas atómicas de la carpeta temática se reúnen en un markdown estructurado, conservando las citas de fuente
- 2 Graficar** (opcional) — Generar los gráficos con Python/matplotlib (paleta de marca, ≥180 DPI, rutas relativas)
- 3 Maquetar** — Escribir el `.typ`: tokens de color, componentes reutilizables, portada, índice, secciones, header y footer
- 4 Compilar y verificar** — `typst compile doc.typ doc.pdf` y revisar visualmente en Preview antes de publicar
- 5 Publicar y versionar** — Copiar `.typ` y `.pdf` al destino; conservar las versiones anteriores (`-v1`, `-v2...`), nunca sobrescribir

Anatomía, tokens y componentes

Todo entregable sigue la misma estructura: **portada** (título, versión, fecha, “ELABORADO POR”) → **índice** → **secciones numeradas** con headings H1/H2/H3 consistentes → **footer** con numeración automática → **pie final** de autoría.

La paleta se declara como tokens al inicio del .typ. **Regla de separación de marcas:** un entregable de la metodología BRAIN usa su paleta propia (navy/coral); un entregable de un proyecto cliente usa la paleta de ese cliente. Nunca se mezclan. Los componentes reutilizables (callout, coral-callout, hero-box, pillar-box, estilos de heading y header/footer) se copian sin modificar entre documentos de la misma serie.

Atribución

Todos los documentos generados llevan atribución del autor en tres lugares:

Lugar	Contenido
Portada	Campo “ELABORADO POR” con nombre completo del autor
Footer de cada página	Nombre · © · año
Pie final	Nota de autoría completa con © y restricciones de distribución

07 — Principios de diseño

Principio	Descripción
Fidelidad	Nunca inventar datos, reglas o pasos. Si la fuente no lo dice, no se afirma. Se marca estado: borrador lo que requiere verificación.
Atomicidad	Una idea por nota. Si una nota crece hasta cubrir dos conceptos que podrían cambiar independientemente, se parte en dos.
Citación obligatoria	Toda nota cita su fuente al pie: archivo de origen y número de página. Sin fuente, la nota no está terminada.
Enlace liberal	Se enlaza aunque la nota destino no exista aún. Un enlace huérfano es un marcador de trabajo futuro válido, no un error.
Inmutabilidad de fuentes	Los archivos en Markdown/ son sagrados — nunca se modifican. Son la fuente de verdad inmutable.
Ciclo vivo	El cerebro no se construye una sola vez. Todo aprendizaje nuevo vuelve al cerebro como nota nueva o actualización.
Escala incremental	No es necesario atomizar todo el corpus antes de usar el sistema. Se expande on-demand, cuando se necesita.

08 — Caso de aplicación (ilustrativo)

A modo de ejemplo **hipotético** — no corresponde a ningún cliente ni proyecto real — considérese la aplicación de BRAIN a un proyecto de integración en un sector regulado de alta densidad documental.

El corpus

Tipo	Volumen	Contenido
Normativo/técnico	Decenas de miles de líneas	Especificaciones de APIs, reglamentos, lineamientos de UX y ciberseguridad, catálogos de errores
Talleres funcionales	Una serie	Flujos de proceso documentados
Estratégico	1 documento	Research & Discovery: mercado, competidores, oportunidades identificadas

El resultado

Varios cientos de notas atómicas organizadas en carpetas temáticas, con un grafo de miles de conexiones. Permitiría responder preguntas complejas sobre cumplimiento normativo, generar código que ya cumple las especificaciones técnicas, y producir documentos estratégicos estructurados.

Un caso así valida la metodología en su escala más exigente: corpus multidimensional (técnico + normativo + estratégico), múltiples versiones de documentos, y necesidad de trazabilidad completa entre cada línea de código y la regla que la justifica.

Nota: La metodología es aplicable a cualquier proyecto con características similares — no es específica de ningún sector ni país.

09 — Autoría e IP

BRAIN es una metodología original desarrollada por **Sebastian Dominguez**.

El nombre, la estructura de cuatro pilares, el pipeline de generación de documentos, los patrones de atomización y los componentes de diseño son propiedad intelectual de Sebastian Dominguez.

Las **aplicaciones de la metodología** son instancias de uso — el vault generado pertenece al contexto de cada proyecto, pero la metodología en sí pertenece a su creador.

Elaborado por **Sebastian Dominguez** · BRAIN v1.0 · Junio 2026
© 2026 Sebastian Dominguez — Todos los derechos reservados
Contacto: dcsebastianc@gmail.com